ASSIGNMENT NO. 7

AIM: Parent process writes a pathname of a file (the contents of the file are desired) on one pipe to be read by child process and child process writes the contents of the file on second pipe to be read by parent process and displays on standard output**.**

Objectives

1. To understand inter-process communication (IPC).
2. To understand the working of pipes in UNIX/Linux.
3. To demonstrate communication between a parent and child process.
4. To implement full-duplex communication using two pipes.

Theory

Pipe

A pipe is an IPC mechanism that allows data to flow from one process to another.

A pipe is created using:

```
pipe(fd);
```

where:

```
fd[0] → Read end
fd[1] → Write end
```

A normal pipe provides **one-way communication**. To achieve communication in both directions, **two pipes** are required.

Full-Duplex Communication

- **Pipe 1:** Parent → Child
- **Pipe 2:** Child → Parent

Program 7A1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "fifo1"
#define FIFO2 "fifo2"
#define BUFFER_SIZE 1024

int main()
{
    int fd1, fd2;
    char pathname[256];
    char buffer[BUFFER_SIZE];
    int n;

    // Create two FIFOs
    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Enter pathname of file: ");
    scanf("%255s", pathname);

    /* Open FIFO1 for writing */
    fd1 = open(FIFO1, O_WRONLY);

    // Send pathname to child
    write(fd1, pathname, strlen(pathname) + 1);

    close(fd1);

    /* Open FIFO2 for reading */
    fd2 = open(FIFO2, O_RDONLY);

    printf("\nContents of the file:\n");
    printf("----------------------\n");

    // Read contents sent by child
    while ((n = read(fd2, buffer, BUFFER_SIZE - 1)) > 0)
    {
        buffer[n] = '\0';
        printf("%s", buffer);
    }

    close(fd2);

    // Remove FIFOs
    unlink(FIFO1);
    unlink(FIFO2);

    return 0;}
```

Program 7A2:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "fifo1"
#define FIFO2 "fifo2"
#define BUFFER_SIZE 1024

int main()
{
    int fd1, fd2, file;
    char pathname[256];
    char buffer[BUFFER_SIZE];
    int n;

    // Create two FIFOs
    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    /* Open FIFO1 for reading */
    fd1 = open(FIFO1, O_RDONLY);

    // Read pathname from parent
    read(fd1, pathname, sizeof(pathname));

    close(fd1);

    printf("Child received pathname: %s\n", pathname);

    // Open the requested file
    file = open(pathname, O_RDONLY);

    if (file == -1)
    {
        perror("File opening failed");
        exit(1);
    }
    /* Open FIFO2 for writing */
    fd2 = open(FIFO2, O_WRONLY);

    // Read file and send contents to parent
    while ((n = read(file, buffer, BUFFER_SIZE)) > 0)
    {
        write(fd2, buffer, n);
    }
    close(file);
    close(fd2);

    return 0;
}
```

ASSIGNMENT NO. 8

Disk Scheduling Algorithms

Implement the C program for Disk Scheduling Algorithms: SSTF, SCAN, C-Look considering the initial head position moving away from the spindle.

Objective:

Disk scheduling algorithms manage how data is read from and written to a computer's hard disk. These algorithms help determine the order in which disk read and write requests are processed.

- Disk scheduling is also known as I/O Scheduling.
- The main goals of disk scheduling are to optimize the performance of disk operations, reduce the time it takes to access data and improve overall system efficiency.
- Common disk scheduling methods include First-Come, First-Served (FCFS), Shortest Seek Time First (SSTF), SCAN, C-SCAN, LOOK, and C-LOOK.

Importance of Disk Scheduling in Operating System

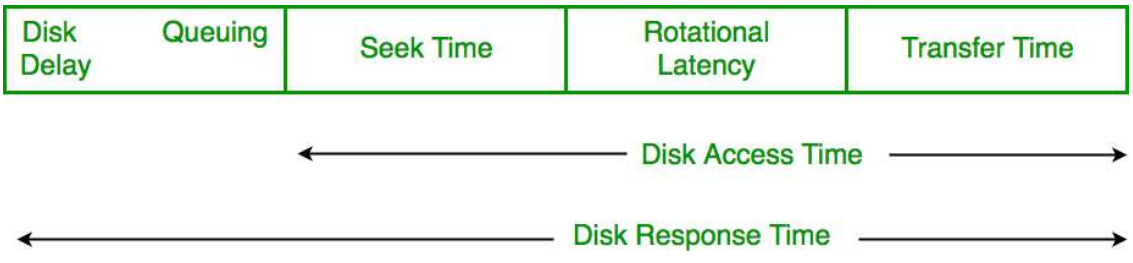
- Multiple I/O requests may arrive by different processes and only one I/O request can be served at a time by the disk controller. Thus other I/O requests need to wait in the waiting queue and need to be scheduled.
- Two or more requests may be far from each other so this can result in greater disk arm movement.
- Hard drives are one of the slowest parts of the computer system and thus need to be accessed in an efficient manner.

Key Terms Associated with Disk Scheduling

- **Seek Time**: Time taken to move the disk arm to the track where data is located.
- **Rotational Latency**: Time taken for the desired sector to rotate under the read/write head.
- **Transfer Time**: Time taken to actually read or write the data, depending on disk speed and data size.

Disk Access Time = Seek Time + Rotational Latency + Transfer Time

*Total Seek Time = Total head Movement * Seek Time*



Disk Response Time

- **Response Time**: The time a request waits before its I/O operation starts.
- **Average Response Time**: The mean waiting time of all requests.
- **Variance in Response Time**: How much individual waiting times differ from the average.

Goals of Disk Scheduling Algorithms

- Minimize Seek Time
- Maximize Throughput
- Minimize Latency
- Ensuring Fairness
- Efficiency in Resource Utilization

Disk Scheduling Algorithms

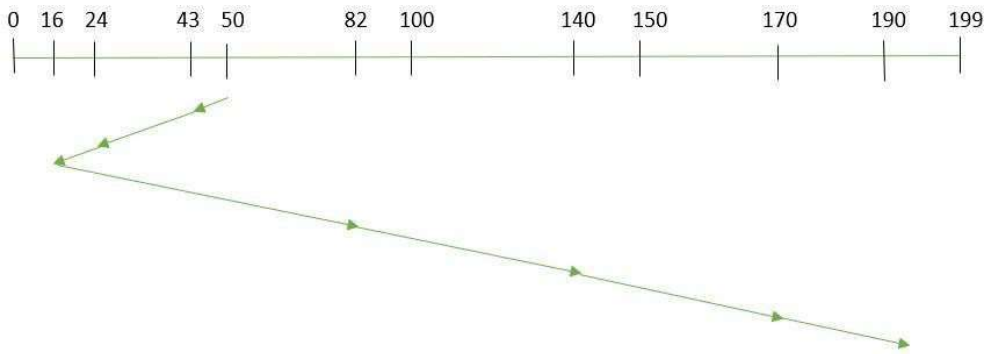
There are several Disk Several Algorithms. We will discuss in detail each one of them.

- FCFS (First Come First Serve)
- SSTF (Shortest Seek Time First)
- SCAN
- C-SCAN
- LOOK
- C-LOOK
- RSS (Random Scheduling)
- **LIFO (Last-In First-Out)**
- **N-STEP SCAN**
- F-SCAN

1 SSTF (Shortest Seek Time First)

In SSTF (Shortest Seek Time First), requests having the shortest seek time are executed first. So, the seek time of every request is calculated in advance in the queue and then they are scheduled according to their calculated seek time. As a result, the request near the disk arm will get executed first. SSTF is certainly an improvement over FCFS as it decreases the average response time and increases the throughput of the system. Let us understand this with the help of an example.

Example:



Shortest Seek Time First

Suppose the order of request is- (82,170,43,140,24,16,190) and current position of Read/Write head is: 50

total overhead movement (total distance covered by the disk arm) =
(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) =208

Advantages of Shortest Seek Time First

Here are some of the advantages of Shortest Seek Time First.

- The average Response Time decreases
- Throughput increases

Disadvantages of Shortest Seek Time First

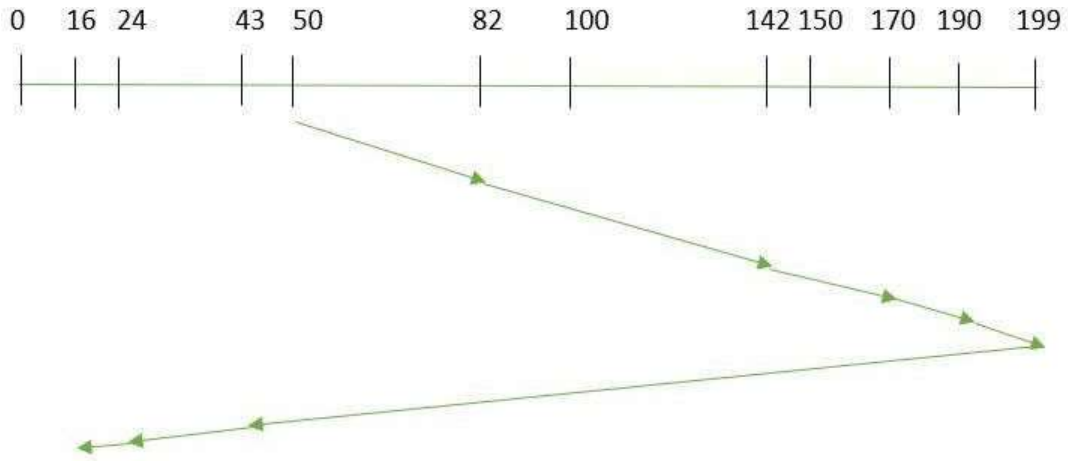
Here are some of the disadvantages of Shortest Seek Time First.

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF favors only some requests

2. SCAN

In the SCAN algorithm the disk arm moves in a particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path. So, this algorithm works as an elevator and is hence also known as an **elevator algorithm.** As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

Example:



SCAN Algorithm

Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as

= (199-50) + (199-16) = 332

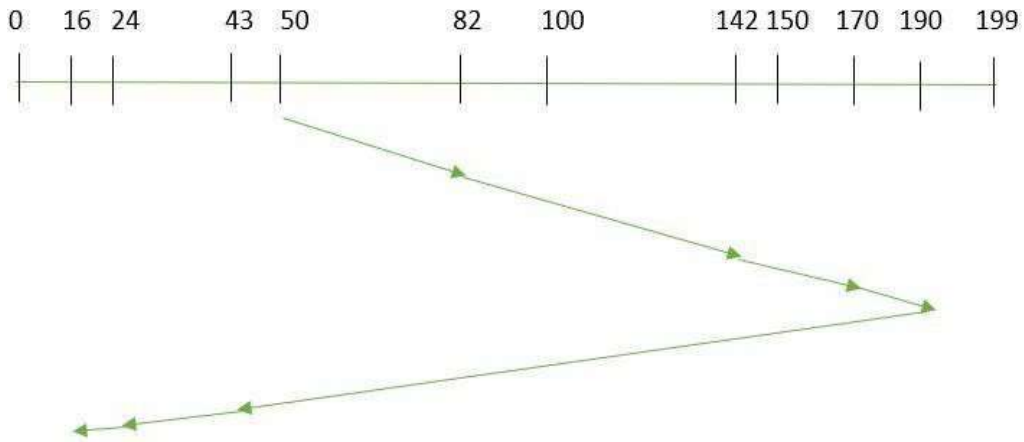
Advantages of SCAN Algorithm
Here are some of the advantages of the SCAN Algorithm.

- High throughput
- Low variance of response time
- Average response time

Disadvantages of SCAN Algorithm: Long waiting time for requests for locations just visited by disk arm

5. LOOK

LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.
Example:



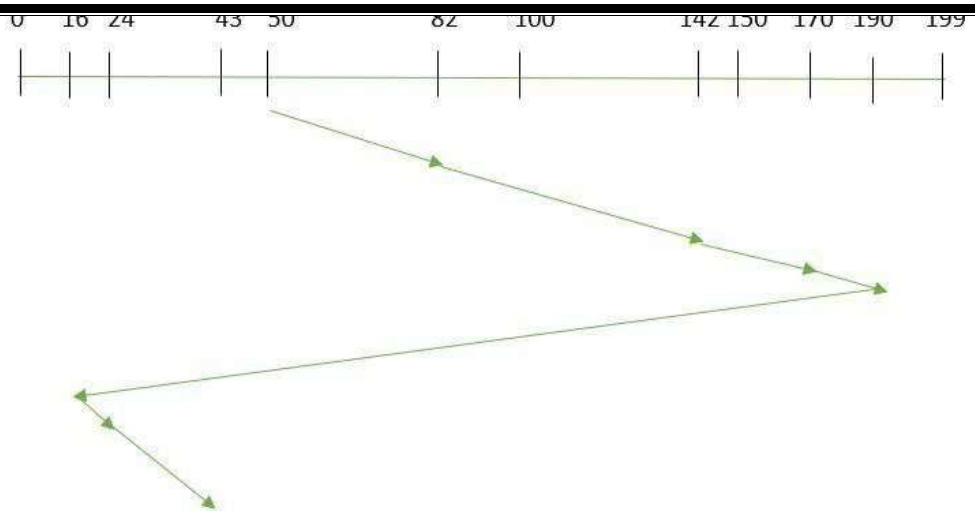
LOOK Algorithm

Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".
So, the total overhead movement (total distance covered by the disk arm) is calculated as:
= (190-50) + (190-16) = 314
Advantages

- **Reduced Unnecessary Movement:** The disk arm only goes as far as the last request in each direction, avoiding travel to the disk's physical end (unlike SCAN).
- **Faster Response:** Less head movement leads to quicker service for requests.

6. C-LOOK

As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm. In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.
Example: Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value"



C-LOOK

So, the total overhead movement (total distance covered by the disk arm) is calculated as
= (190-50) + (190-16) + (43-16) = 341

Advantages:

- **Uniform Wait Time:** Requests are serviced in a circular manner, so waiting times are more predictable and fair.
- **Reduced Head Movement**: The arm only goes as far as the last request in one direction, then jumps back, saving time compared to C-SCAN.

Program 8:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

/* Function to calculate absolute difference */
int distance(int a, int b)
{
    return abs(a - b);
}

/* ---------------- SSTF ---------------- */
void SSTF(int requests[], int n, int head)
{
    int visited[MAX] = {0};
    int total = 0;
    int current = head;

    printf("\nSSTF Sequence: %d", current);

    for (int i = 0; i < n; i++)
    {
        int min = 999999;
        int index = -1;

        for (int j = 0; j < n; j++)
        {
```

```
 if (!visited[j])
            {
                int d = distance(current, requests[j]);

                if (d < min)
                {
                    min = d;
                    index = j;
                }
            }
        }

        visited[index] = 1;
        total += min;
        current = requests[index];

        printf(" -> %d", current);
    }

    printf("\nTotal Head Movement = %d cylinders\n", total);
}

/* ---------------- SCAN ---------------- */
void SCAN(int requests[], int n, int head, int disk_size)
{
    int arr[MAX + 2];
    int count = 0;
    int total = 0;
    int current = head;

    /* Add requests */
    for (int i = 0; i < n; i++)
    {
        arr[count++] = requests[i];
    }

    /* Add initial head */
    arr[count++] = head;

    /* Add end of disk because head moves away from spindle */
    arr[count++] = disk_size - 1;

    /* Sort */
    for (int i = 0; i < count - 1; i++)
    {
        for (int j = i + 1; j < count; j++)
        {
            if (arr[i] > arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
```

```
    /* Find head position */
    int pos = 0;

    for (int i = 0; i < count; i++)
    {
        if (arr[i] == head)
        {
            pos = i;
            break;
        }
    }

    printf("\nSCAN Sequence: %d", head);

    /* Move away from spindle: toward higher cylinders */
    for (int i = pos + 1; i < count; i++)
    {
        total += distance(current, arr[i]);
        current = arr[i];

        /* Don't print disk end as a request, but it is part of movement */
        if (arr[i] != disk_size - 1)
            printf(" -> %d", current);
    }

    printf(" -> %d", disk_size - 1);

    /* Reverse direction */
    for (int i = pos - 1; i >= 0; i--)
    {
        total += distance(current, arr[i]);
        current = arr[i];

        printf(" -> %d", current);
    }

    printf("\nTotal Head Movement = %d cylinders\n", total);
}

/* ---------------- C-LOOK ---------------- */
void CLOOK(int requests[], int n, int head)
{
    int arr[MAX];
    int total = 0;
    int current = head;
    int pos = 0;

    /* Copy requests */
    for (int i = 0; i < n; i++)
    {
        arr[i] = requests[i];
    }

    /* Sort requests */
    for (int i = 0; i < n - 1; i++)
```

```
        {
            for (int j = i + 1; j < n; j++)
            {
                if (arr[i] > arr[j])
                {
                    int temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }









      /* Find first request greater than or equal to head */
        while (pos < n && arr[pos] < head)
            pos++;

        printf("\nC-LOOK Sequence: %d", head);

        /* Move away from spindle: toward higher cylinders */
        for (int i = pos; i < n; i++)
        {
            total += distance(current, arr[i]);
            current = arr[i];

            printf(" -> %d", current);
        }

        /*
         * C-LOOK jumps from the highest request
         * to the lowest request.
         */
        if (pos > 0 && pos < n)
        {
            total += distance(current, arr[0]);
            current = arr[0];

            printf(" -> %d", current);

            for (int i = 1; i < pos; i++)
            {
                total += distance(current, arr[i]);
                current = arr[i];

                printf(" -> %d", current);
            }
        }

        printf("\nTotal Head Movement = %d cylinders\n", total);
    }

    /* ---------------- MAIN ---------------- */
    int main()
    {
        int requests[MAX];
        int n, head, disk_size;
```

```
    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    printf("Enter disk request queue:\n");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &requests[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter total number of cylinders: ");
    scanf("%d", &disk_size);

    printf("\nInitial head position = %d", head);
    printf("\nInitial direction = Away from spindle");
    printf("\nDirection = Toward higher cylinder numbers\n");




    SSTF(requests, n, head);

    SCAN(requests, n, head, disk_size);

    CLOOK(requests, n, head);

    return 0;
}
```

CONCLUSION: Thus, we studied disk scheduling.